

1. Basic data exploration & data cleaning

```
In [2]: # This Python 3 environment comes with many helpful analytics libraries i
# It is defined by the kaggle/python Docker image: https://github.com/kag
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import matplotlib.pyplot as plt
import seaborn as sns

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) wil

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) th
# You can also write temporary files to /kaggle/temp/, but they won't be

/kaggle/input/netflix-scaler-business-cs/netflix.csv
```

```
In [3]: df = pd.read_csv("/kaggle/input/netflix-scaler-business-cs/netflix.csv")

df.head()
```

Out [3]:

	show_id	type	title	director	cast	country	date_added	release_y
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2021
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thabane...	South Africa	September 24, 2021	2021
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021

In [4]: df.info

```

Out[4]: <bound method DataFrame.info of
title      director \
0          s1      Movie    Dick Johnson Is Dead    Kirsten Johnson
1          s2      TV Show      Blood & Water      NaN
2          s3      TV Show      Ganglands    Julien Leclercq
3          s4      TV Show    Jailbirds New Orleans    NaN
4          s5      TV Show      Kota Factory      NaN
...      ...      ...
8802      s8803      Movie      Zodiac    David Fincher
8803      s8804      TV Show      Zombie Dumb    NaN
8804      s8805      Movie      Zombieland    Ruben Fleischer
8805      s8806      Movie      Zoom    Peter Hewitt
8806      s8807      Movie      Zubaan    Mozez Singh

cast      country
\
0      NaN    United States
1    Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...    South Africa
2    Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...    NaN
3      NaN    NaN
4    Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...    India
...      ...      ...
8802    Mark Ruffalo, Jake Gyllenhaal, Robert Downey J...    United States
8803      NaN    NaN
8804    Jesse Eisenberg, Woody Harrelson, Emma Stone, ...    United States
8805    Tim Allen, Courteney Cox, Chevy Chase, Kate Ma...    United States
8806    Vicky Kaushal, Sarah-Jane Dias, Raaghav Chanan...    India

date_added  release_year  rating  duration \
0    September 25, 2021      2020    PG-13    90 min
1    September 24, 2021      2021    TV-MA    2 Seasons
2    September 24, 2021      2021    TV-MA    1 Season
3    September 24, 2021      2021    TV-MA    1 Season
4    September 24, 2021      2021    TV-MA    2 Seasons
...      ...      ...
8802    November 20, 2019      2007      R    158 min
8803      July 1, 2019      2018    TV-Y7    2 Seasons
8804    November 1, 2019      2009      R    88 min
8805    January 11, 2020      2006      PG    88 min
8806      March 2, 2019      2015    TV-14    111 min

listed_in \
0      Documentaries
1    International TV Shows, TV Dramas, TV Mysteries
2    Crime TV Shows, International TV Shows, TV Act...
3      Docuseries, Reality TV
4    International TV Shows, Romantic TV Shows, TV ...
...      ...
8802      Cult Movies, Dramas, Thrillers
8803      Kids' TV, Korean TV Shows, TV Comedies
8804      Comedies, Horror Movies
8805      Children & Family Movies, Comedies
8806      Dramas, International Movies, Music & Musicals

description
0    As her father nears the end of his life, filmm...
1    After crossing paths at a party, a Cape Town t...
2    To protect his family from a powerful drug lor...
3    Feuds, flirtations and toilet talk go down amo...
4    In a city of coaching centers known to train I...

```

```

...
8802 A political cartoonist, a crime reporter and a...
8803 While living alone in a spooky town, a young g...
8804 Looking to survive in a world taken over by zo...
8805 Dragged from civilian life, a former superhero...
8806 A scrappy but poor boy worms his way into a ty...

[8807 rows x 12 columns]>

```

```
In [5]: print(df.shape)
```

```
(8807, 12)
```

Modifying and cleaning data

```
In [6]: df['date_added'] = pd.to_datetime(df['date_added'], errors='coerce')
df['release_year'] = df['release_year'].astype('Int64')
df['type'] = df['type'].astype('category')

df.head(2)
```

```
Out[6]:
```

	show_id	type	title	director	cast	country	date_added	release_year
--	---------	------	-------	----------	------	---------	------------	--------------

0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	2021-09-25	2021
---	----	-------	-------------------------	-----------------	-----	---------------	------------	------

1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thabane...	South Africa	2021-09-24	2021
---	----	---------	---------------	-----	---	--------------	------------	------

Exploring missing data

```
In [7]: missing = df.isna().sum()
missing_percent = (missing / len(df) * 100).round(2)
pd.concat([missing, missing_percent], axis=1, keys=['missing_count', 'miss
```

Out [7]:

	missing_count	missing_pct
director	2634	29.91
country	831	9.44
cast	825	9.37
date_added	98	1.11
rating	4	0.05
duration	3	0.03
show_id	0	0.00
type	0	0.00
title	0	0.00
release_year	0	0.00
listed_in	0	0.00
description	0	0.00

Exploring individual fields

In [8]: `df['release_year'].describe()`

Out [8]:

count	8807.0
mean	2014.180198
std	8.819312
min	1925.0
25%	2013.0
50%	2017.0
75%	2019.0
max	2021.0

Name: release_year, dtype: Float64

In [9]: `df['type'].value_counts()`

Out [9]:

type	
Movie	6131
TV Show	2676

Name: count, dtype: int64

In [10]: `df['listed_in'].value_counts().head(20)`

```
Out[10]: listed_in
Dramas, International Movies      362
Documentaries                    359
Stand-Up Comedy                  334
Comedies, Dramas, International Movies 274
Dramas, Independent Movies, International Movies 252
Kids' TV                        220
Children & Family Movies        215
Children & Family Movies, Comedies 201
Documentaries, International Movies 186
Dramas, International Movies, Romantic Movies 180
Comedies, International Movies    176
Comedies, International Movies, Romantic Movies 152
Dramas                          138
Dramas, International Movies, Thrillers 134
Action & Adventure, Dramas, International Movies 132
Action & Adventure              128
International TV Shows, TV Dramas 121
Comedies, Dramas, Independent Movies 116
Crime TV Shows, International TV Shows, TV Dramas 110
Comedies                        110
Name: count, dtype: int64
```

```
In [11]: df['country'].fillna('Unknown').value_counts().head(10)
```

```
Out[11]: country
United States      2818
India              972
Unknown            831
United Kingdom     419
Japan              245
South Korea        199
Canada             181
Spain              145
France             124
Mexico             110
Name: count, dtype: int64
```

```
In [12]: # Unique count
{
    'titles': df['title'].unique(),
    'directors': df['director'].unique(dropna=True),
    'actors': df['cast'].unique(dropna=True),
    'countries': df['country'].unique(dropna=True)
}
```

```
Out[12]: {'titles': 8807, 'directors': 4528, 'actors': 7692, 'countries': 748}
```

```
In [13]: # top genre

genres_series = df['listed_in'].dropna().str.split(',').explode().str.strip()
genres_series.value_counts().head(20)
```

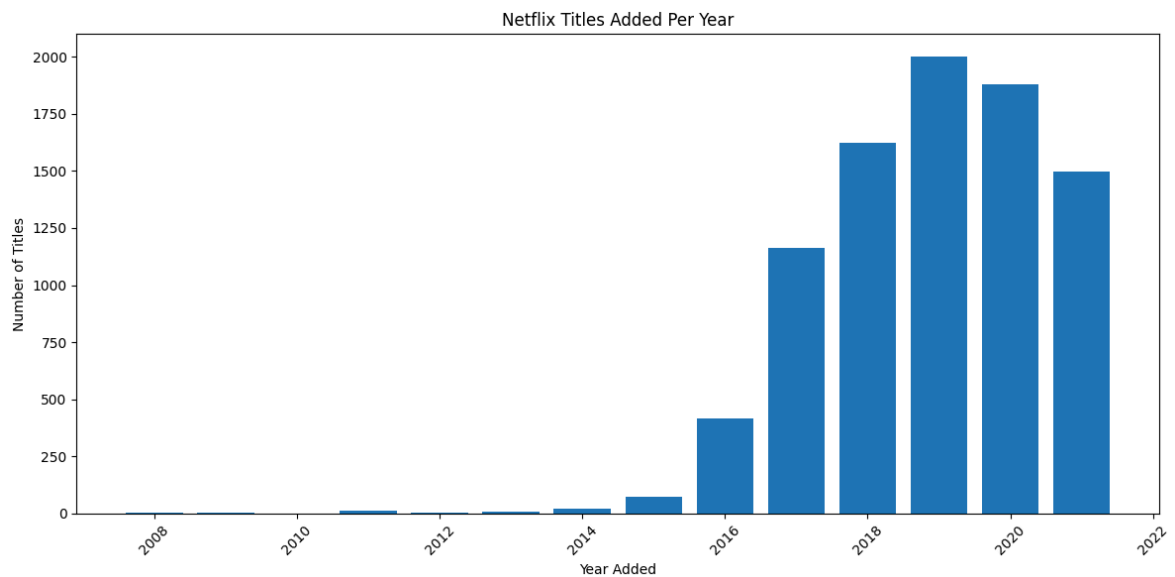
```
Out[13]: listed_in
International Movies      2752
Dramas                   2427
Comedies                  1674
International TV Shows   1351
Documentaries             869
Action & Adventure       859
TV Dramas                 763
Independent Movies        756
Children & Family Movies  641
Romantic Movies           616
TV Comedies               581
Thrillers                 577
Crime TV Shows            470
Kids' TV                  451
Docuseries                395
Music & Musicals          375
Romantic TV Shows        370
Horror Movies             357
Stand-Up Comedy          343
Reality TV                255
Name: count, dtype: int64
```

Netflix growth through years

```
In [14]: df['added_year'] = df['date_added'].dt.year
df['added_month'] = df['date_added'].dt.to_period('M')
year_counts = df['added_year'].value_counts().sort_index() # new titles
year_counts
```

```
Out[14]: added_year
2008.0      2
2009.0      2
2010.0      1
2011.0     13
2012.0      3
2013.0     10
2014.0     23
2015.0     73
2016.0    418
2017.0   1164
2018.0   1625
2019.0   1999
2020.0   1878
2021.0   1498
Name: count, dtype: int64
```

```
In [15]: # Plot
plt.figure(figsize=(12,6))
plt.bar(year_counts.index, year_counts.values)
plt.xlabel("Year Added")
plt.ylabel("Number of Titles")
plt.title("Netflix Titles Added Per Year")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Plot observations

- There has been a rapid increase in growth since 2015
- The growth has been steady from 2015 - 2019
- After 2019 the growth has slowed down

Comparing TV shows and movies

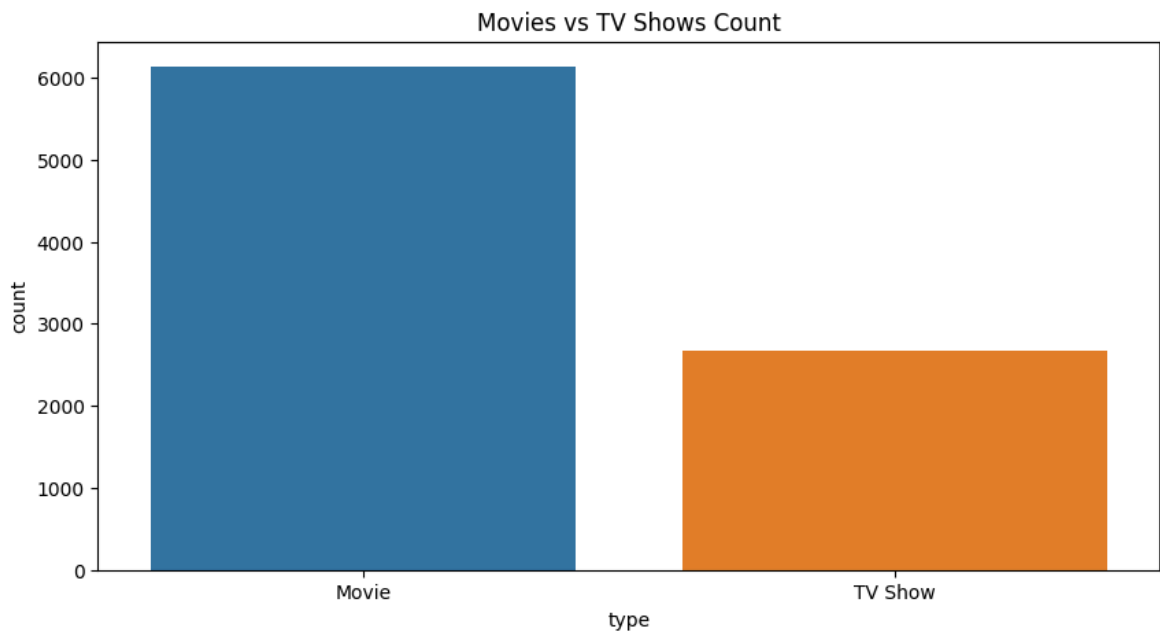
```
In [16]: df['type'].value_counts()
```

```
Out[16]: type
Movie      6131
TV Show    2676
Name: count, dtype: int64
```

```
In [17]: plt.figure(figsize=(10,5))
sns.countplot(data=df, x='type')
plt.title('Movies vs TV Shows Count')
plt.show()
```

/usr/local/lib/python3.11/dist-packages/seaborn/categorical.py:641: Future Warning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
grouped_vals = vals.groupby(grouper)
```



Plot summary

- There are twice as many Movies produced as TV shows.
- Since Netflix started with movies business and movies are easier comparatively to produce.

Recommendations

- Increase investment in TV shows to balance the catalogue and drive long-term engagement.
- Continue producing movies but focus on strategic, high-impact titles rather than volume.

```
In [18]: df['type'].value_counts()
(df['type'].value_counts(normalize=True) * 100).round(2)
yearly_counts = (
    df.groupby(['added_year', 'type'])
      .size()
      .reset_index(name='count')
)
yearly_counts
```

```
/tmp/ipykernel_47/3497637176.py:4: FutureWarning: The default of observed=
False is deprecated and will be changed to True in a future version of pan
das. Pass observed=False to retain current behavior or observed=True to ad
opt the future default and silence this warning.
  df.groupby(['added_year', 'type'])
```

Out [18]:

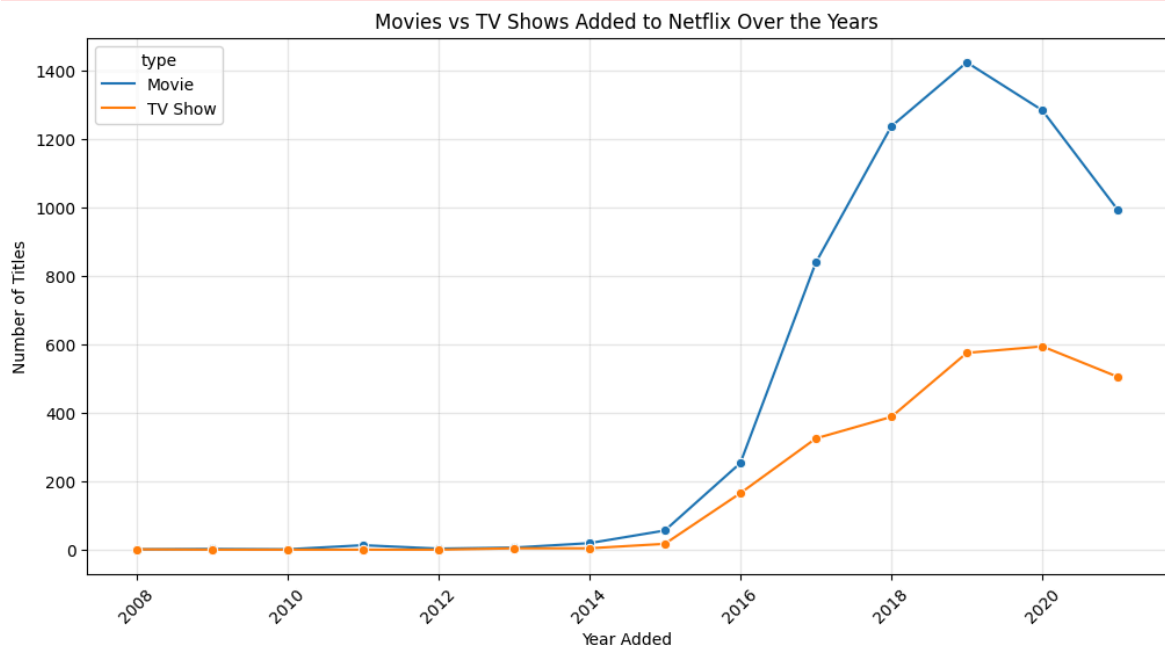
	added_year	type	count
0	2008.0	Movie	1
1	2008.0	TV Show	1
2	2009.0	Movie	2
3	2009.0	TV Show	0
4	2010.0	Movie	1
5	2010.0	TV Show	0
6	2011.0	Movie	13
7	2011.0	TV Show	0
8	2012.0	Movie	3
9	2012.0	TV Show	0
10	2013.0	Movie	6
11	2013.0	TV Show	4
12	2014.0	Movie	19
13	2014.0	TV Show	4
14	2015.0	Movie	56
15	2015.0	TV Show	17
16	2016.0	Movie	253
17	2016.0	TV Show	165
18	2017.0	Movie	839
19	2017.0	TV Show	325
20	2018.0	Movie	1237
21	2018.0	TV Show	388
22	2019.0	Movie	1424
23	2019.0	TV Show	575
24	2020.0	Movie	1284
25	2020.0	TV Show	594
26	2021.0	Movie	993
27	2021.0	TV Show	505

In [19]: `plt.figure(figsize=(12,6))`

```
sns.lineplot(  
    data=yearly_counts,  
    x='added_year',  
    y='count',  
    hue='type',  
    marker='o'  
)
```

```
plt.title('Movies vs TV Shows Added to Netflix Over the Years')
plt.xlabel('Year Added')
plt.ylabel('Number of Titles')
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.show()
```

```
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1057: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
  grouped_data = data.groupby(
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1075: FutureWarning: When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.
  data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1075: FutureWarning: When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.
  data_subset = grouped_data.get_group(pd_key)
```



Best time to launch a TV show

```
In [20]: df['month_num'] = df['date_added'].dt.month

tv = df[df['type'] == 'TV Show']
monthly_counts = tv['month_num'].value_counts().sort_index()

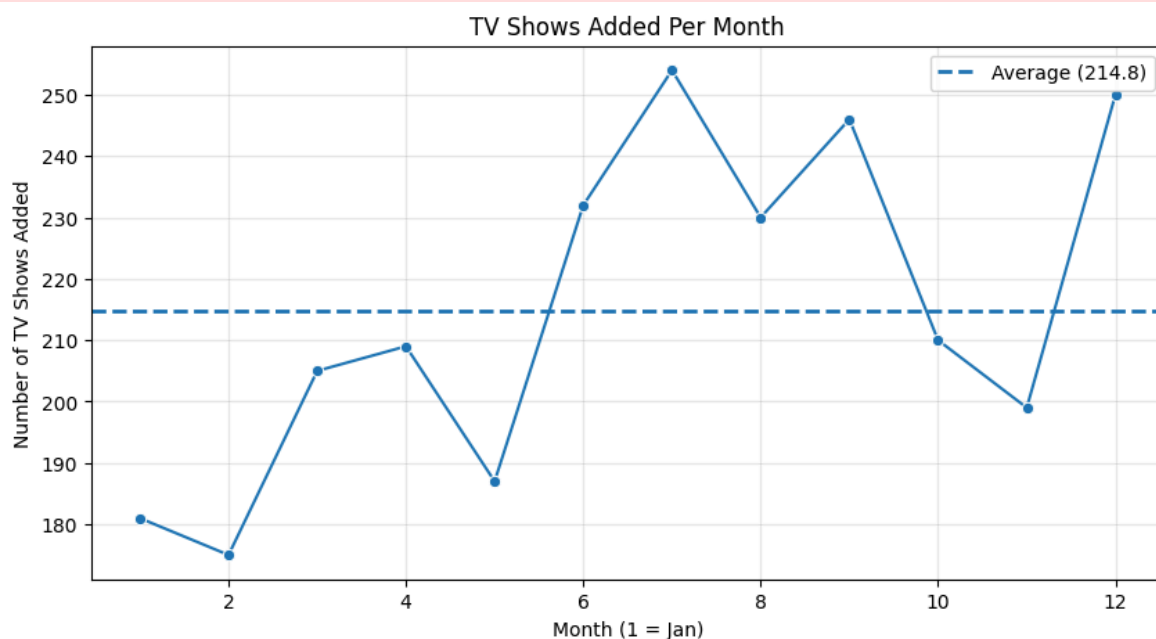
In [21]: plt.figure(figsize=(10,5))
sns.lineplot(x=monthly_counts.index, y=monthly_counts.values, marker='o')

# Calculate average
avg = monthly_counts.mean()

# Add horizontal average line
plt.axhline(avg, linestyle='--', linewidth=2, label=f'Average ({avg:.1f})')

plt.title('TV Shows Added Per Month')
plt.xlabel('Month (1 = Jan)')
plt.ylabel('Number of TV Shows Added')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
 with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
 with pd.option_context('mode.use_inf_as_na', True):



Plot Summary :

- In June , July , August , September and December are the highest movies released.

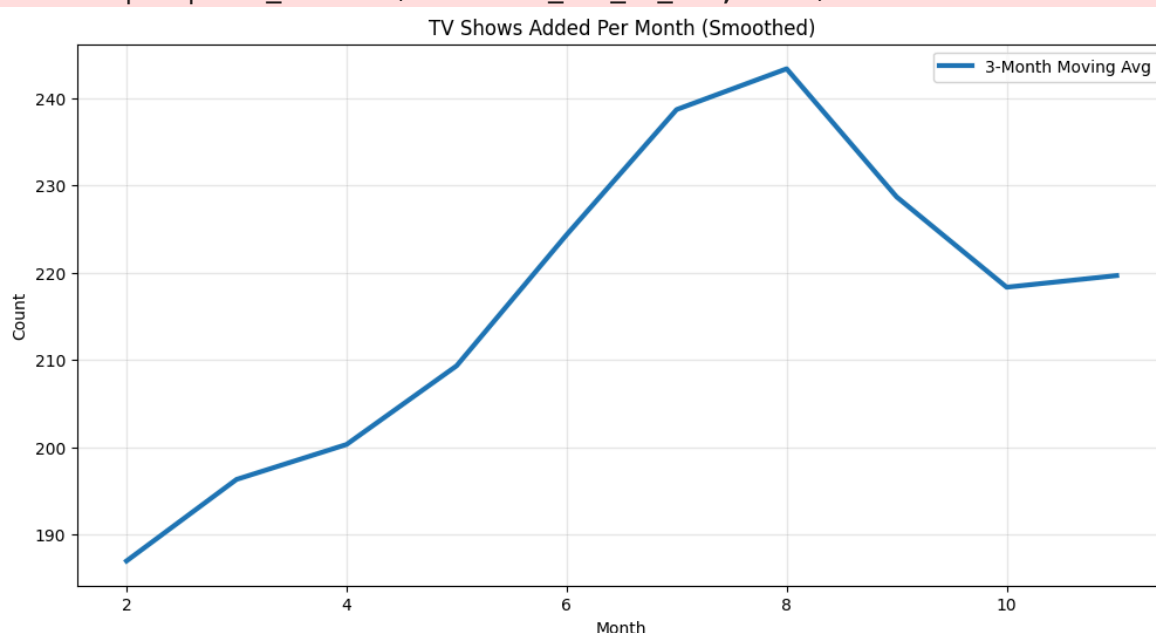
```
In [22]: monthly_counts_ma = monthly_counts.rolling(window=3, center=True).mean()

plt.figure(figsize=(12,6))
```

```
sns.lineplot(x=monthly_counts.index, y=monthly_counts_ma.values, label="3-Month Moving Avg")

plt.title("TV Shows Added Per Month (Smoothed)")
plt.xlabel("Month")
plt.ylabel("Count")
plt.grid(alpha=0.3)
plt.legend()
plt.show()
```

```
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



```
In [23]: plt.figure(figsize=(12,6))

sns.lineplot(x=monthly_counts.index, y=monthly_counts.values, label="Actual")
sns.lineplot(x=monthly_counts.index,
              y=monthly_counts.rolling(3, center=True).mean(),
              label="3-Month Moving Avg", linewidth=3)

# Winter (Dec=12, Jan=1, Feb=2)
plt.axvspan(12, 13, color='lightblue', alpha=0.2)
plt.axvspan(1, 3, color='lightblue', alpha=0.2, label='Winter')

# Spring
plt.axvspan(3, 6, color='lightgreen', alpha=0.2, label='Spring')

# Summer
plt.axvspan(6, 9, color='gold', alpha=0.2, label='Summer')

# Autumn
plt.axvspan(9, 12, color='lightsalmon', alpha=0.2, label='Autumn')

plt.title("TV Shows Added Per Month with Seasonality Shading")
plt.xlabel("Month")
```

```
plt.ylabel("Number of TV Shows Added")
plt.xticks(range(1,13))
plt.grid(alpha=0.3)
plt.legend()
plt.show()
```

/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.

with pd.option_context('mode.use_inf_as_na', True):

/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.

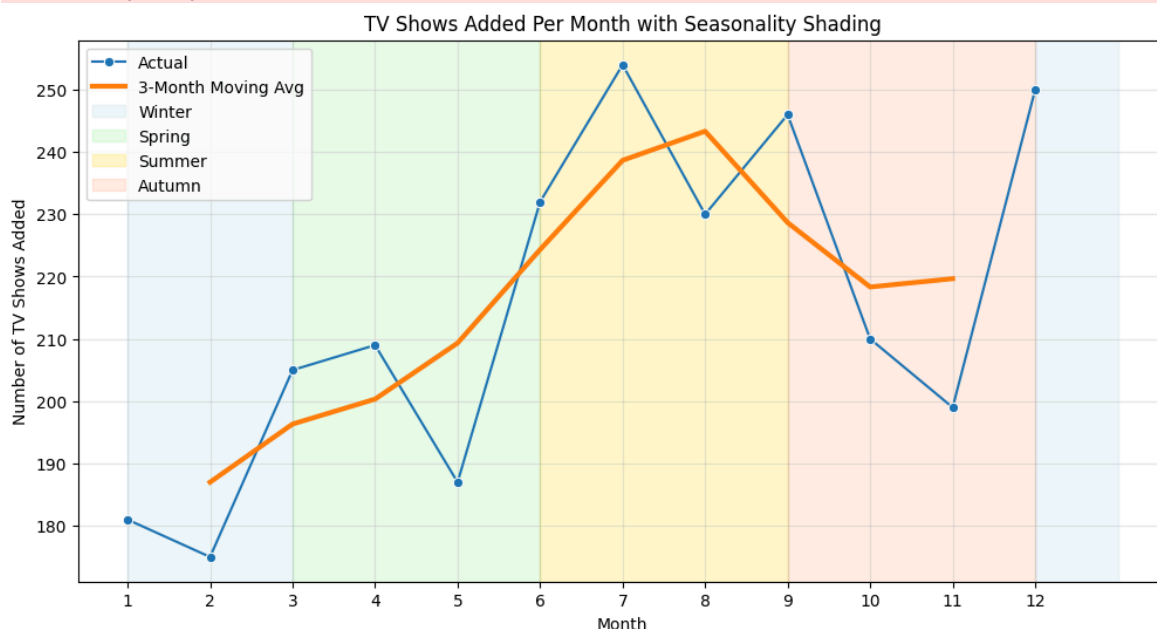
with pd.option_context('mode.use_inf_as_na', True):

/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.

with pd.option_context('mode.use_inf_as_na', True):

/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.

with pd.option_context('mode.use_inf_as_na', True):



Plot summary :

- There is a clear seasonlaity trend in the summer season.
- The trend is likely caused by summer holidays
- Autumn end always has a steady increaase riding the build up for summer.
- Winter is the worst time to release a TV shows.

Recommendations :

- Prioritize TV-show launches during summer months (June–August).
- Avoid major releases in winter and focus instead on content buildup for the following summer.

Analysis of actors/directors of different types of shows/movies.

```
In [24]: def explode_column(df, col):
df = df.copy()
df[col] = df[col].fillna("").str.split(",")
df = df.explode(col)
df[col] = df[col].str.strip()
df = df[df[col] != ""]
return df
```

```
In [25]: actors_df = explode_column(df, "cast")
directors_df = explode_column(df, "director")

actors_df.head(3)
```

```
Out [25]:
```

	show_id	type	title	director	cast	country	date_added	release_year	
1	s2	TV Show	Blood & Water	NaN	Ama Qamata	South Africa	2021-09-24	2021	
1	s2	TV Show	Blood & Water	NaN	Khosi Ngema	South Africa	2021-09-24	2021	
1	s2	TV Show	Blood & Water	NaN	Gail Mabalane	South Africa	2021-09-24	2021	

```
In [26]: directors_df.head(3)
```

Out [26]:

	show_id	type	title	director	cast	country	date_added	release_ye
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	2021-09-25	20
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	2021-09-24	20
5	s6	TV Show	Midnight Mass	Mike Flanagan	Kate Siegel, Zach Gilford, Hamish Linklater, H...	NaN	2021-09-24	20

```
In [27]: top_actors = (
    actors_df.groupby(['type', 'cast'])
            .size()
            .reset_index(name='count')
        )

top_actors_movies = top_actors[top_actors['type'] == 'Movie'].sort_values
top_actors_tv = top_actors[top_actors['type'] == 'TV Show'].sort_values('
top_actors_movies
```

/tmp/ipykernel_47/2933957274.py:2: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
actors_df.groupby(['type', 'cast'])
```

Out [27]:

	type	cast	count
2832	Movie	Anupam Kher	42
30489	Movie	Shah Rukh Khan	35
24215	Movie	Naseeruddin Shah	32
25424	Movie	Om Puri	30
845	Movie	Akshay Kumar	30
16697	Movie	Julie Tejawani	28
25782	Movie	Paresh Rawal	28
1773	Movie	Amitabh Bachchan	28
28974	Movie	Rupa Bhimani	27
4527	Movie	Boman Irani	27
17221	Movie	Kareena Kapoor	25
29600	Movie	Samuel L. Jackson	22
765	Movie	Ajay Devgn	21
27355	Movie	Rajesh Kava	21
24752	Movie	Nicolas Cage	20

In [28]: top_actors_tv

Out [28]:

	type	cast	count
69030	TV Show	Takahiro Sakurai	25
72319	TV Show	Yuki Kaji	19
53262	TV Show	Junichi Suwabe	17
37135	TV Show	Ai Kayano	17
43709	TV Show	Daisuke Ono	17
72300	TV Show	Yuichi Nakamura	16
53199	TV Show	Jun Fukuyama	15
72180	TV Show	Yoshimasa Hosoya	15
44270	TV Show	David Attenborough	14
57433	TV Show	Mamoru Miyano	13
72187	TV Show	Yoshitsugu Matsuoka	13
69054	TV Show	Takehito Koyasu	13
53536	TV Show	Kana Hanazawa	13
71156	TV Show	Vincent Tong	13
49258	TV Show	Hiroshi Kamiya	13

```
In [29]: top_directors = (
    directors_df.groupby(['type', 'director'])
               .size()
               .reset_index(name='count')
    )

top_directors_movies = top_directors[top_directors['type'] == 'Movie'].so
top_directors_tv = top_directors[top_directors['type'] == 'TV Show'].sort

top_directors_movies, top_directors_tv
```

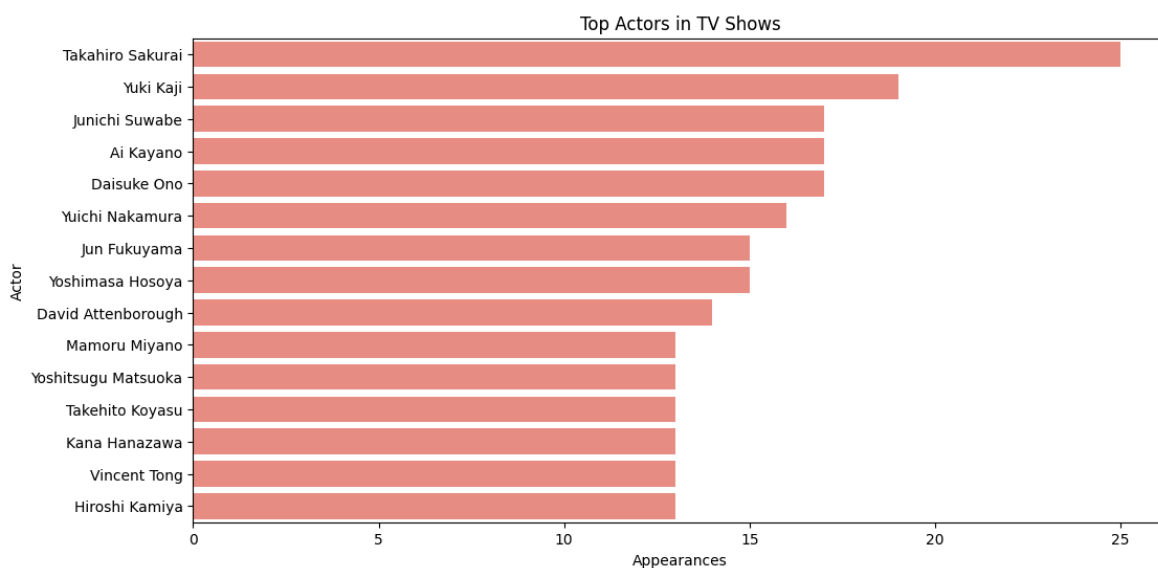
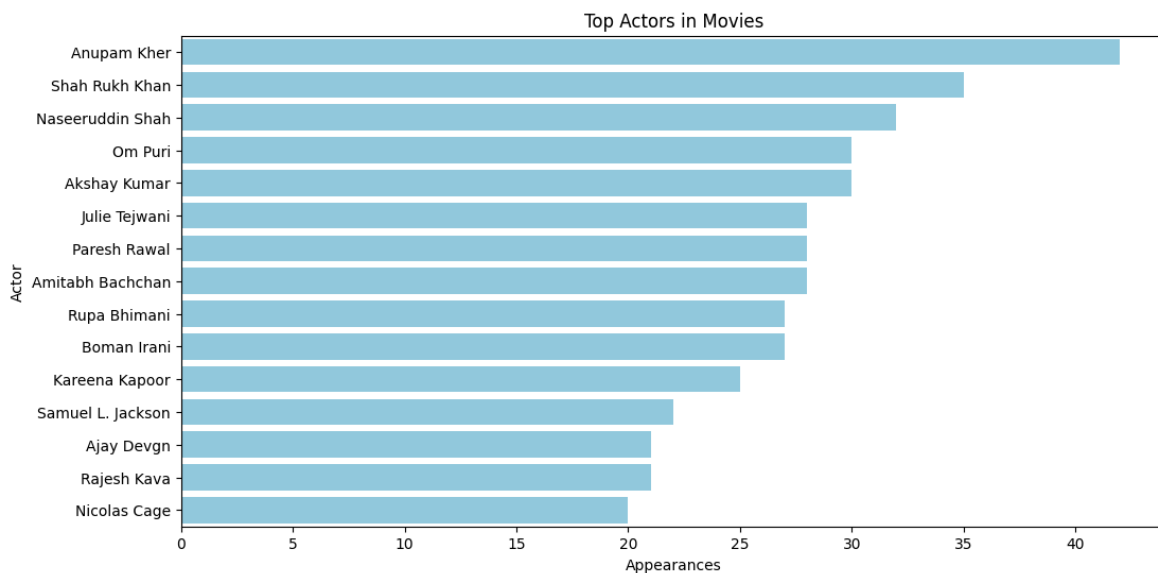
/tmp/ipykernel_47/1163673733.py:2: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
directors_df.groupby(['type', 'director'])
```

```
Out[29]: (      type      director  count
3749  Movie    Rajiv Chilaka    22
1906  Movie      Jan Suter    21
3800  Movie    Raúl Campos    19
4457  Movie    Suhas Kadav    16
1954  Movie      Jay Karas    15
2866  Movie    Marcus Raboy    15
755   Movie  Cathy Garcia-Molina    13
2945  Movie    Martin Scorsese    12
1951  Movie      Jay Chapman    12
4941  Movie    Youssef Chahine    12
4425  Movie    Steven Spielberg    11
1217  Movie    Don Michael Paul    10
1075  Movie      David Dhawan     9
4243  Movie    Shannon Hartman     9
4952  Movie    Yılmaz Erdoğan     9,
      type      director  count
5138  TV Show  Alastair Fothergill     3
7443  TV Show      Ken Burns     3
9353  TV Show      Stan Lathan     2
7756  TV Show      Lynn Novick     2
6779  TV Show    Iginio Straffi     2
7335  TV Show      Jung-ah Im     2
6520  TV Show  Gautham Vasudev Menon     2
6748  TV Show      Hsu Fu-chun     2
9267  TV Show      Shin Won-ho     2
7085  TV Show      Joe Berlinger     2
8892  TV Show    Rob Seidenglanz     2
8140  TV Show      Miguel Conde     1
8125  TV Show    Michel Tikhomiroff     1
9342  TV Show    Spencer Williams     1
9347  TV Show      Srijit Mukherji     1)
```

```
In [30]: plt.figure(figsize=(12,6))
sns.barplot(
    data=top_actors_movies,
    x='count', y='cast', color='skyblue'
)
plt.title("Top Actors in Movies")
plt.xlabel("Appearances")
plt.ylabel("Actor")
plt.show()
```

```
plt.figure(figsize=(12,6))
sns.barplot(
    data=top_actors_tv,
    x='count', y='cast', color='salmon'
)
plt.title("Top Actors in TV Shows")
plt.xlabel("Appearances")
plt.ylabel("Actor")
plt.show()
```



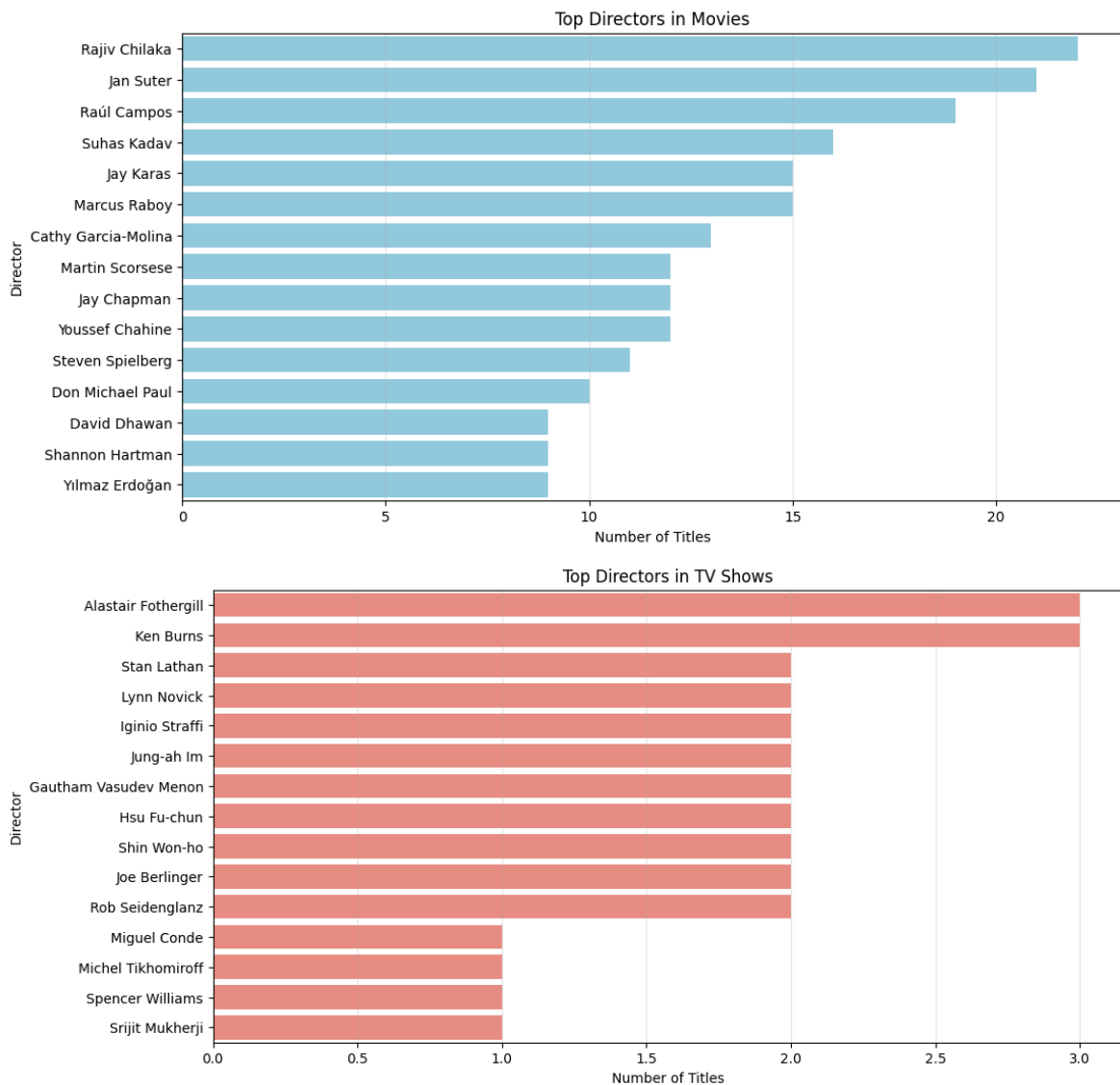
```
In [31]: plt.figure(figsize=(12,6))
sns.barplot(
    data=top_directors_movies,
    x='count',
    y='director',
    color='skyblue'
)

plt.title("Top Directors in Movies")
plt.xlabel("Number of Titles")
plt.ylabel("Director")
plt.grid(axis='x', alpha=0.3)
plt.show()

plt.figure(figsize=(12,6))
```

```
sns.barplot(
    data=top_directors_tv,
    x='count',
    y='director',
    color='salmon'
)

plt.title("Top Directors in TV Shows")
plt.xlabel("Number of Titles")
plt.ylabel("Director")
plt.grid(axis='x', alpha=0.3)
plt.show()
```



Plot Summary

- Movies are dominated by high-output Indian and American talent, whereas TV shows draw heavily from Japanese and international creators.
- Directors show a sharper divide between Movies and TV than actors do.

Recommendations

- Invest more in Japanese TV content, especially anime and documentary-style series.

- Strengthen movie partnerships with Indian and American film industries.

Netflix has more focus on TV Shows than movies in recent years

```
In [32]: yearly = df.groupby(['added_year', 'type']).size().reset_index(name='count_yearly')
```

```
/tmp/ipykernel_47/4126094758.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
```

```
yearly = df.groupby(['added_year', 'type']).size().reset_index(name='count')
```

Out [32]:

	added_year	type	count
0	2008.0	Movie	1
1	2008.0	TV Show	1
2	2009.0	Movie	2
3	2009.0	TV Show	0
4	2010.0	Movie	1
5	2010.0	TV Show	0
6	2011.0	Movie	13
7	2011.0	TV Show	0
8	2012.0	Movie	3
9	2012.0	TV Show	0
10	2013.0	Movie	6
11	2013.0	TV Show	4
12	2014.0	Movie	19
13	2014.0	TV Show	4
14	2015.0	Movie	56
15	2015.0	TV Show	17
16	2016.0	Movie	253
17	2016.0	TV Show	165
18	2017.0	Movie	839
19	2017.0	TV Show	325
20	2018.0	Movie	1237
21	2018.0	TV Show	388
22	2019.0	Movie	1424
23	2019.0	TV Show	575
24	2020.0	Movie	1284
25	2020.0	TV Show	594
26	2021.0	Movie	993
27	2021.0	TV Show	505

```

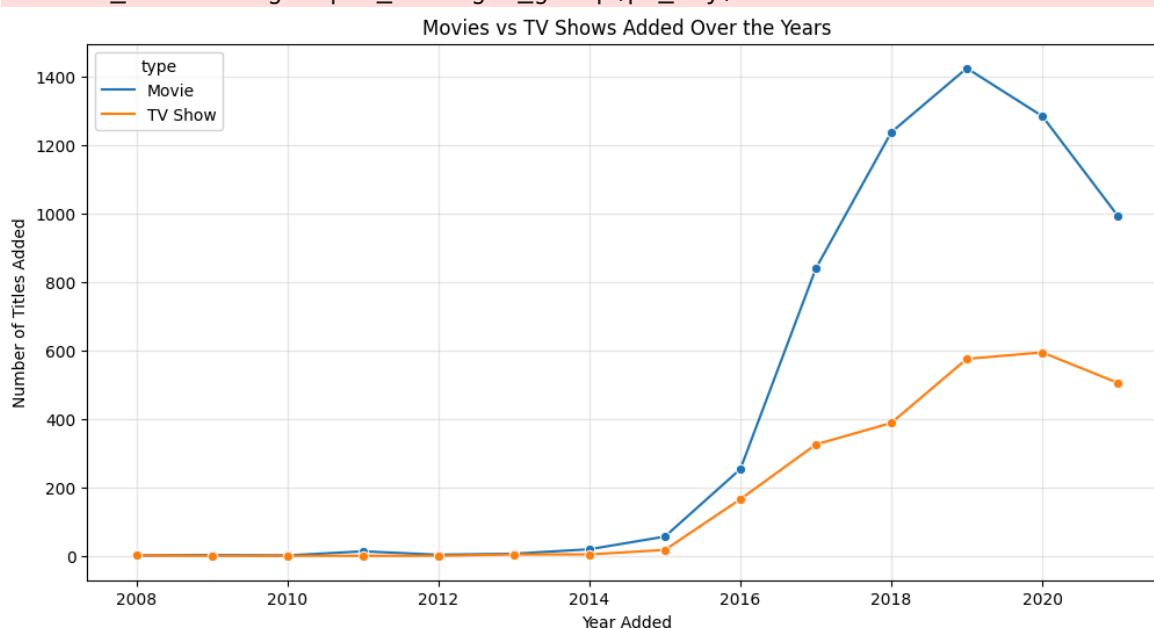
In [33]: plt.figure(figsize=(12,6))
sns.lineplot(data=yearly, x='added_year', y='count', hue='type', marker='o')
plt.title('Movies vs TV Shows Added Over the Years')
plt.xlabel('Year Added')
plt.ylabel('Number of Titles Added')
plt.grid(alpha=0.3)
plt.show()

```

```

/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1057: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
  grouped_data = data.groupby(
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1075: FutureWarning: When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.
  data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1075: FutureWarning: When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.
  data_subset = grouped_data.get_group(pd_key)

```



Plot Summary

- The trend clearly shows that Netflix has increased its focus on TV Shows in recent years.
- Movies dominate the early part of the dataset, but TV Shows grow much faster in the later years, especially after 2016.
- In the most recent years of the dataset, TV Shows either match or exceed Movies in yearly additions.

This suggests a shift by Netflix toward producing more long-form series, which help increase watch time and subscriber retention.

Understanding what content is available in different countries

```
In [41]: # Explode the country column
df_country = df.copy()
df_country['country'] = df_country['country'].fillna("Unknown").str.split
df_country = df_country.explode('country')
df_country['country'] = df_country['country'].str.strip()

df_country['country']
```

```
Out[41]: 0      United States
1      South Africa
2      Unknown
3      Unknown
4      India
...
8802   United States
8803   Unknown
8804   United States
8805   United States
8806   India
Name: country, Length: 10850, dtype: object
```

```
In [42]: country_counts = df_country['country'].value_counts().head(20)
print(country_counts)
```

```
country
United States    3690
India            1046
Unknown          831
United Kingdom   806
Canada           445
France           393
Japan            318
Spain            232
South Korea      231
Germany          226
Mexico           169
China            162
Australia        160
Egypt            117
Turkey           113
Hong Kong        105
Nigeria          103
Italy            100
Brazil            97
Argentina         91
Name: count, dtype: int64
```

```
In [45]: top_countries = country_counts.index[:10] # top 10

plot_data = country_type[country_type['country'].isin(top_countries)]

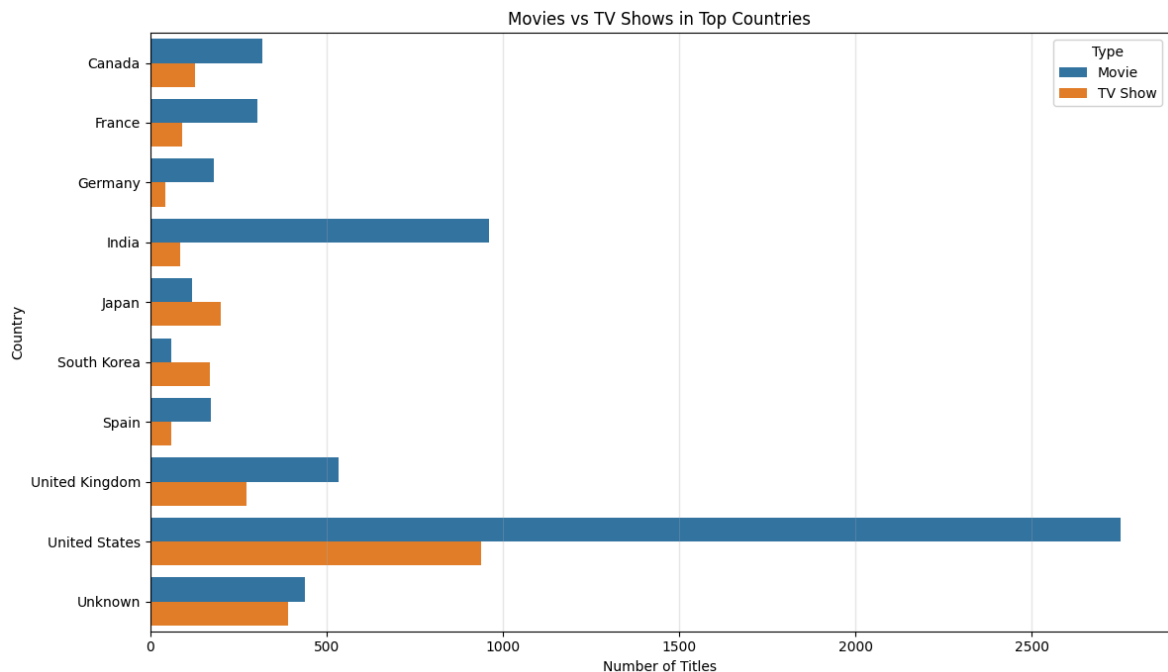
plt.figure(figsize=(12,7))
sns.barplot(
    data=plot_data,
    x='count',
```

```

y='country',
hue='type',
orient='h'
)

plt.title("Movies vs TV Shows in Top Countries")
plt.xlabel("Number of Titles")
plt.ylabel("Country")
plt.grid(axis='x', alpha=0.3)
plt.legend(title="Type")
plt.tight_layout()
plt.show()

```



```

In [44]: country_type = (
df_country.groupby(['country', 'type'])
            .size()
            .reset_index(name='count')
)

```

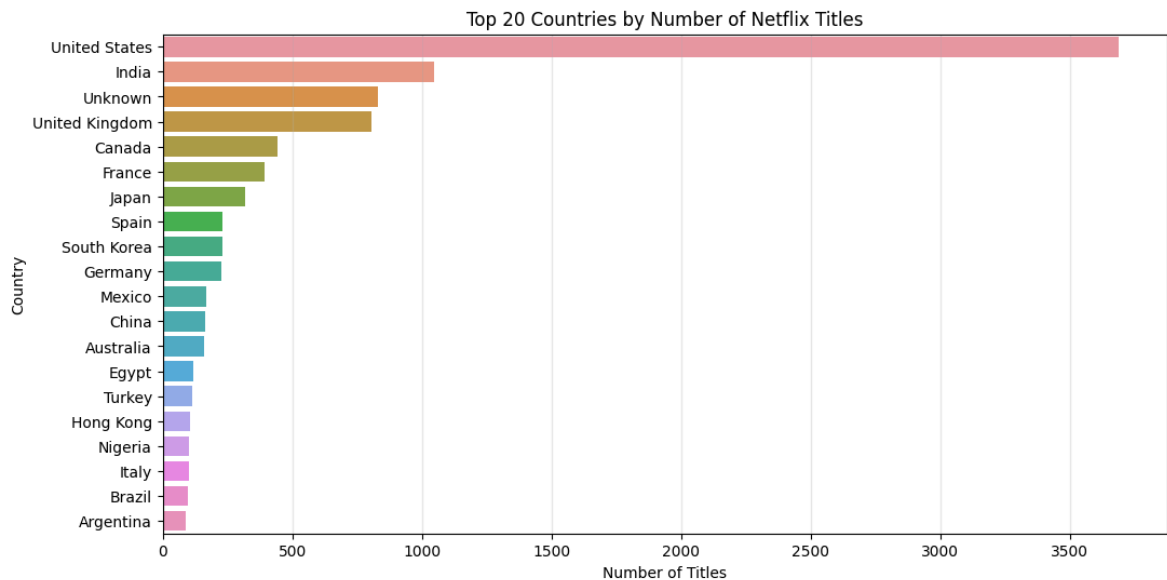
/tmp/ipykernel_47/1939061156.py:2: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
df_country.groupby(['country', 'type'])
```

```

In [43]: plt.figure(figsize=(12,6))
sns.barplot(x=country_counts.values, y=country_counts.index)
plt.title("Top 20 Countries by Number of Netflix Titles")
plt.xlabel("Number of Titles")
plt.ylabel("Country")
plt.grid(axis="x", alpha=0.3)
plt.show()

```



Country summaries :

- USA has a balanced movies and TV show production
- India is mostly movies
- Japan mostly TV shows which is because of the anime.
- South Korea is mostly TV shows because of K-drama
- UK is mostly balanced but slightly TV heavy
- Canada is TV heavy.

Plot Summary

- The United States dominates Netflix's catalogue, contributing far more titles than any other country.
- India is the second-largest content contributor, driven mainly by its high-volume movie industry.
- Japan, South Korea, Spain, and the UK supply strong TV-focused content, reflecting active regional production hubs.